

PASSAGE Watch Backend Requirements

API specification for watch state control, location tracking, verification, authentication, and offline behaviour.

Product Area	PASSAGE Watch
Document Type	Backend Requirements
Primary Goal	Let the backend control the watch screen while the watch automatically sends location data.

1. Purpose

The watch is mostly non-interactive. The backend controls which screen appears, and the watch automatically sends location data to the backend.

In practice, the child should not need to swipe, choose screens, or manually control the ride flow. The backend sends the current state; the watch displays it.

2. Required Endpoints

2.1 Get Watch State

Method	Endpoint	Authorization
GET	/api/watch/state/{deviceId}	Bearer DEVICE_TOKEN

Controls the screen displayed on the watch.

Example response

```
{
  "state": "DRIVER_NEARBY",
  "tripId": "trip_98765",
  "message": "Please get ready",
  "trackingIntervalSeconds": 30
}
```

Supported states

IDLE_READY	RIDE_ASSIGNED
DRIVER_NEARBY	AWAITING_VERIFICATION
PICKUP_CONFIRMED	IN_TRANSIT
DROPOFF_CONFIRMED	SOS_ACTIVE
LOW_BATTERY	OFFLINE

2.2 Send Watch Location

Method	Endpoint	Authorization
POST	/api/watch/location	Bearer DEVICE_TOKEN

Example request

```
{
  "deviceId": "watch_12345",
  "tripId": "trip_98765",
  "lat": 0.3476,
  "lng": 32.5825,
  "accuracy": 8.5,
  "speed": 4.2,
  "bearing": 120,
  "battery": 86,
  "timestamp": "2026-06-08T08:00:00Z"
}
```

Suggested frequency

- No trip: every 2-5 minutes
- Ride assigned / driver nearby: every 30-60 seconds
- Active trip: every 10-30 seconds
- SOS: every 5-10 seconds
- Drop-off: send one final location

2.3 Send Watch Event

Method	Endpoint	Authorization
POST	/api/watch/event	Bearer DEVICE_TOKEN

Used for SOS and low battery events.

Example request

```
{
  "deviceId": "watch_12345",
  "tripId": "trip_98765",
  "eventType": "SOS_TRIGGERED",
  "battery": 64,
  "timestamp": "2026-06-08T08:15:00Z"
}
```

2.4 Request Verification Token

Method	Endpoint	Authorization
POST	/api/watch/verification-token	Bearer DEVICE_TOKEN

Example request

```
{
  "deviceId": "watch_12345",
  "tripId": "trip_98765"
}
```

Example response

```
{
  "verificationToken": "signed_short_lived_token",
  "expiresAt": "2026-06-08T08:30:00Z"
}
```

The backend creates the secure token. The watch converts it into a QR image.

2.5 Verify Watch QR

Used by the rider app, not the watch.

Method	Endpoint	Authorization
POST	/api/trips/verify-watch	Bearer RIDER_TOKEN

Example request

```
{
  "tripId": "trip_98765",
  "verificationToken": "signed_short_lived_token",
  "action": "PICKUP"
}
```

Backend verification checks

- Token is valid and unexpired
- Token belongs to the watch and trip
- Rider is assigned to the trip
- Token has not already been used

After successful verification, the backend changes the watch state to PICKUP_CONFIRMED.

3. Automatic Screen Flow

1	IDLE_READY	->
2	RIDE_ASSIGNED	->
3	DRIVER_NEARBY	->
4	AWAITING_VERIFICATION	->
5	PICKUP_CONFIRMED	->
6	IN_TRANSIT	->
7	DROPOFF_CONFIRMED	->
8	IDLE_READY	

The watch polls the state endpoint and automatically changes screens. The child does not swipe or choose screens.

4. Authentication

Each watch needs the following credentials:

deviceId	deviceToken
----------	-------------

All watch requests use this authorization header:

```
Authorization: Bearer DEVICE_TOKEN
```

Unknown or disabled devices must be rejected.

5. Offline Behaviour

If there is no internet connection, the watch should behave safely and preserve trip data.

- Save location updates locally
- Display the offline screen
- Upload saved locations when internet returns
- Do not lose active trip information

A separate heartbeat endpoint is not required. Battery and connectivity information can be included in location updates.

6. Implementation Notes

- The backend is the source of truth for watch screen state.
- The watch should poll the state endpoint and update its UI automatically.
- Location frequency should change based on trip urgency and safety state.
- QR tokens should be short-lived, signed, single-use, and linked to a specific watch and trip.
- Offline storage is required so that location history is not lost during network gaps.